```
"use client";

import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function DashboardAssets({
  clientId,
  programId,
  embedded,
}: {
  clientId: number | null;
  programId: number | null;
  /** When true, render only inner content (no outer card) for use inside
a parent card */
  embedded?: boolean;
```

```
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  //  Load files when sidebar selection changes
  useEffect(() => {
    if (!clientId || !programId) {
      setFiles([]);
      return;
    }

    setLoading(true);

    fetch(`${API}/api/attachments/?client=${clientId}&program=${programId}`)
      .then((r) => r.json())
      .then(setFiles)
      .finally(() => setLoading(false));
  }, [clientId, programId]);

  // If nothing selected in sidebar
  if (!clientId || !programId) {
```

```
    const fallback = (
      <p className="text-sm text-slate-500 dark:text-slate-400">
        Select client and program from sidebar to view assets.
      </p>
    );
    if (embedded) return fallback;
    return (
      <div className="p-4 border rounded bg-white dark:bg-slate-
900">
        {fallback}
      </div>
    );
  }
  function isDirectVideoUrl(url: string) {
    const lower = url.toLowerCase();
    return (
      lower.endsWith(".mp4") ||
      lower.endsWith(".webm") ||
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();
```

```
    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` :
null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
```

```
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return https://www.loom.com/embed/${pathParts[shareIndex + 1]};
      }

      const embedIndex = pathParts.findIndex((part) => part === "embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) return;

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
```

```
      window.open(file.external_url, "_blank", "noopener,noreferrer");
      return;
    }

    setVideoPopup({
      title: file.file_name,
      videoUrl: file.external_url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }
  // Inner item style matching Program Statistics cards: rounded-lg bg-
white border border-slate-100
  const fileItem = (f: Attachment) => (
    <div key={f.id} className="rounded-lg bg-white p-2.5 sm:p-3 text-
sm border border-slate-100 dark:bg-slate-900 dark:border-slate-800
min-w-0">
      <div className="text-xs font-medium text-slate-900 dark:text-
slate-50 break-words">{f.file_name}</div>
      {f.description && (
        <DescriptionDisplay html={f.description} className="text-xs text-
slate-500 dark:text-slate-400 mt-0.5" />
      )}
          {f.asset_type === "file" && f.sharepoint_url ? (
        <a
          href={f.sharepoint_url}
          target="_blank"
          rel="noopener noreferrer"
          className="inline-flex items-center gap-1 mt-1.5 text-xs font-
```

```
medium text-emerald-600 dark:text-emerald-400 hover:underline"
        >
          View file
          <span aria-hidden>→</span>
        </a>
      ) : f.asset_type === "url" && f.external_url ? (
        isPlayableVideoUrl(f.external_url) ? (
          <button
            onClick={() => openUrlVideo(f)}
            className="inline-flex items-center gap-1 mt-1.5 px-2.5 py-1
text-xs font-medium rounded-lg bg-emerald-600 text-white hover:bg-
emerald-700 transition"
          >
            ▶️ Play video
          </button>
        ) : (
          <a
            href={f.external_url}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 mt-1.5 text-xs font-
medium text-emerald-600 dark:text-emerald-400 hover:underline"
          >
            Open link
            <span aria-hidden>→</span>
          </a>
        )
      ) : null}
    </div>
```

```
  );

  const listContent = (
    <>
      {loading && (
        <div className="flex flex-col items-center justify-center py-8
text-center">
          <div className="rounded-full bg-slate-100 dark:bg-slate-800 p-
3 mb-2">
            <FolderOpen className="h-5 w-5 text-slate-400 dark:text-
slate-500 animate-pulse" aria-hidden />
          </div>
          <p className="text-sm text-slate-500 dark:text-slate-
400">Loading files...</p>
        </div>
      )}
      {files.length === 0 && !loading && (
        <div className="flex flex-col items-center justify-center py-8 px-
4 text-center rounded-lg border border-dashed border-slate-200
dark:border-slate-700 bg-slate-50/50 dark:bg-slate-800/30 min-h-
[120px]">
          <FileX2 className="h-8 w-8 text-slate-300 dark:text-slate-600
mb-2" aria-hidden />
          <p className="text-sm font-medium text-slate-600 dark:text-
slate-400">No files assigned</p>
          <p className="text-xs text-slate-500 dark:text-slate-400 mt-
0.5">Assets will appear here when assigned to this program.</p>
        </div>
      )}
```

```
      {files.length > 0 && !loading && (
        <div className="grid gap-3">
          {files.map((f) => fileItem(f))}
        </div>
      )}
    </>
  );

  if (embedded) {
    return (
      <>
        <UrlVideoPopup
          open={videoPopup != null}
          title={videoPopup?.title || "Video Player"}
          videoUrl={videoPopup?.videoUrl || null}
          embedUrl={videoPopup?.embedUrl || null}
          isDirectVideo={videoPopup?.isDirectVideo || false}
          onClose={() => setVideoPopup(null)}
        />

        <div className="flex flex-col flex-1 min-h-0">
          <div className="flex items-center justify-between flex-shrink-
0">
            <div className="inline-flex items-center gap-2 text-xs font-
medium text-slate-600 dark:text-slate-400">
              <FolderOpen className="h-4 w-4 text-emerald-500 flex-
shrink-0" aria-hidden />
              <span className="break-words">Assigned Assets
({files.length})</span>
```

```
          </div>
        </div>
        <div className="mt-4 flex-1 min-h-0 overflow-y-auto">
          {listContent}
        </div>
      </div>
    </>
  );
}

return (
  <>
    <UrlVideoPopup
      open={videoPopup != null}
      title={videoPopup?.title || "Video Player"}
      videoUrl={videoPopup?.videoUrl || null}
      embedUrl={videoPopup?.embedUrl || null}
      isDirectVideo={videoPopup?.isDirectVideo || false}
      onClose={() => setVideoPopup(null)}
    />

    <section className="mt-6 rounded-xl border border-slate-200
dark:border-slate-700 bg-white dark:bg-slate-900 p-4">
      <h3 className="text-sm font-semibold mb-3 text-slate-900
dark:text-slate-50">
        Assigned Assets ({files.length})
      </h3>
      {listContent}
    </section>
```

```
    </>
  );
}
```

this is my sashboardassets.tsx of feature branch

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
```

```
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  useEffect(() => {
    if (!programId) {
      setFiles([]);
      return;
    }

    async function loadFiles() {
      setLoading(true);

      try {
        const res = await fetch(`${API}/api/attachments/?program=${programId}`);
        const data = await res.json();
        setFiles(data);
      } catch (err) {
        console.error("Failed to load program files", err);
        setFiles([]);
      } finally {
        setLoading(false);
      }
    }

    loadFiles();
  }, [programId, API]);
```

```
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;
```

```
      // already embed URL
      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` :
null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return `https://www.loom.com/embed/${pathParts[shareIndex +
1]}`;
      }

      const embedIndex = pathParts.findIndex((part) => part ===
"embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }
```

```
    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) return;

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}
```

```
  if (!programId) return null;

  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="flex flex-col">
        {/* Header */}
        <div className="flex items-center gap-2 mb-3">
          <div className="flex h-8 w-8 items-center justify-center
rounded-lg bg-gradient-to-br from-sky-500 to-sky-600">
            <FolderOpen className="h-4 w-4 text-white" />
          </div>
          <div>
            <h2 className="text-xs lg:text-sm font-semibold text-slate-
900 dark:text-slate-50">
              Learning Resources
            </h2>
            <p className="text-[9px] lg:text-xs text-slate-600 dark:text-
slate-400">
              Files & materials
```

```
            </p>
          </div>
        </div>

        {/* Content Box */}
        <div className="h-[100px] overflow-y-auto rounded-lg border
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/70
p-2.5">
          {loading ? (
            <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
              <FileText className="mx-auto h-5 w-5 text-slate-400
dark:text-slate-600" />
              <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
                Loading files...
              </p>
            </div>
          ) : files.length === 0 ? (
            <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
              <FileText className="mx-auto h-5 w-5 text-slate-400
dark:text-slate-600" />
              <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
                No files assigned to this program
              </p>
```

```
          </div>
        ) : (
          <div className="space-y-2">
            {files.map((file) => {
              const isVideoUrl =
                file.asset_type === "url" &&
                !!file.external_url &&
                isPlayableVideoUrl(file.external_url);

              return (
                <div
                  key={file.id}
                  className="rounded-lg border border-slate-200
dark:border-slate-800 bg-white dark:bg-slate-900/60 p-2"
                >
                  <div className="flex items-start gap-2">
                    <div className="flex h-7 w-7 flex-shrink-0 items-center
justify-center rounded-md bg-slate-100 dark:bg-slate-800 text-slate-
600 dark:text-slate-300">
                      <FileText className="h-3.5 w-3.5" />
                    </div>

                    <div className="min-w-0 flex-1">
                      <span className="block text-[10px] lg:text-xs font-
medium text-slate-900 dark:text-slate-100 truncate">
                        {file.file_name}
                      </span>

                      {/* Action */}
```

```
                    <div className="mt-1.5">
                     {file.asset_type === "file" && file.sharepoint_url ? (
                      <a
                       href={file.sharepoint_url}
                       target="_blank"
                       rel="noopener noreferrer"
                       className="inline-flex items-center gap-1 text-
[10px] lg:text-xs font-medium text-emerald-600 dark:text-emerald-400
hover:underline"
                      >
                       View file
                       <span aria-hidden>→</span>
                      </a>
                     ) : file.asset_type === "url" && file.external_url ? (
                      isVideoUrl ? (
                       <button
                        type="button"
                        onClick={() => openUrlVideo(file)}
                        className="inline-flex items-center gap-1 px-2.5
py-1 text-[10px] lg:text-xs font-medium rounded-lg bg-emerald-600
text-white hover:bg-emerald-700 transition"
                       >
                        ▶️ Play video
                       </button>
                      ) : (
                       <a
                        href={file.external_url}
                        target="_blank"
                        rel="noopener noreferrer"
```

```
                          className="inline-flex items-center gap-1 text-
[10px] lg:text-xs font-medium text-emerald-600 dark:text-emerald-400
hover:underline"
                        >
                          Open link
                          <span aria-hidden>→</span>
                        </a>
                      )
                    ) : (
                      <span className="text-[10px] lg:text-xs text-slate-
400 dark:text-slate-500">
                        No link available
                      </span>
                    )}
                  </div>
                </div>
              </div>
            </div>
          );
        })}
      </div>
    )}
  </div>
</div>
</>
);
}
```

this is my programattachments of feature branch

```
"use client";
```

```
import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import { CustomDropdown } from "@/components/CustomDropdown";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Option = { id: number; name: string };

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: Option[];
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
```

```
  programId?: number | null;
}) {

  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [clients, setClients] = useState<Option[]>([]);
  const [draftAssignments, setDraftAssignments] = useState<any>({});
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [filteredPrograms, setFilteredPrograms] =
useState<Record<number, Option[]>>({});
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);
  const { toasts, showToast, removeToast } = useToast();

  async function fetchFilesREST() {
    let url = `${API}/api/attachments/`;

    if (clientId || programId) {
      const params = new URLSearchParams();
      if (clientId) params.append("client", String(clientId));
      if (programId) params.append("program", String(programId));
```

```
    url += ?${params.toString()};
  }

  const res = await fetch(url);
  setFiles(await res.json());
}

useEffect(() => {
  fetchFilesREST();
}, [refreshKey, clientId, programId]);

async function fetchClientsGQL() {
  const query = query { clients { id name } };

  const res = await fetch(${API}/api/attachments/graphql/, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ query }),
  });

  const json = await res.json();
  setClients(json.data.clients);
}

useEffect(() => {
  fetchClientsGQL();
}, []);

async function fetchProgramsByClientGQL(fileId: number, clientId:
```

```
number) {
  const query = `
    query($clientId: Int!) {
      programsByClient(clientId: $clientId) {
        id
        name
      }
    }
  `;

  const res = await fetch(`${API}/api/attachments/graphql/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ query, variables: { clientId } }),
  });

  const json = await res.json();

  setFilteredPrograms(prev => ({
    ...prev,
    [fileId]: json.data.programsByClient,
  }));
}

async function saveAssignmentREST(fileId: number) {
  const data = draftAssignments[fileId];

  if (!data?.clients?.length || !data?.programs?.length) {
    showToast("Select Client and Program", "error");
```

```
      return;
    }

    await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });

    fetchFilesREST();
    // Reset draft selection for this file so dropdowns start blank next
time
    setDraftAssignments((prev: any) => {
      const next = { ...prev };
      delete next[fileId];
      return next;
    });
    setOpenMenu(null);
  }

  function requestDeleteFile(fileId: number) {
    setOpenMenu(null);
    setFileIdToDelete(fileId);
  }

  async function confirmDeleteFile() {
    if (fileIdToDelete == null) return;
    const fileId = fileIdToDelete;
    setFileIdToDelete(null);
```

```
  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles(prev => prev.filter(f => f.id !== fileId));
    showToast("File deleted successfully", "success");
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}

function updateDraft(fileId: number, field: string, values: number[]) {
  setDraftAssignments((prev: any) => ({
    ...prev,
    [fileId]: { ...prev[fileId], [field]: values },
  }));
}
  function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
```

```
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link: youtu.be/VIDEO_ID
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
    }

    // YouTube normal link: youtube.com/watch?v=VIDEO_ID
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      // already embed url
      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
```

```
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` :
null;
      }

      // Loom
      if (host.includes("loom.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const shareIndex = pathParts.findIndex((part) => part ===
"share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return `https://www.loom.com/embed/${pathParts[shareIndex +
1]}`;
        }

        const embedIndex = pathParts.findIndex((part) => part ===
"embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }
```

```
function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}
const filteredFiles = files.filter(f =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);
```

```
  const unassigned = filteredFiles.filter(
    f => f.clients.length === 0 && f.programs.length === 0
  );

  const assigned = filteredFiles.filter(
    f => f.clients.length > 0 || f.programs.length > 0
  );

  function FileCard(file: Attachment) {

    const cardRef = useRef<HTMLDivElement>(null);
    const draft = draftAssignments[file.id] || {};
    const disableSave = !draft.clients?.length || !draft.programs?.length;

    useEffect(() => {
      function handleClickOutside(event: MouseEvent) {
        const target = event.target as HTMLElement | null;
        // If click is inside any CustomDropdown overlay, don't close the
assignment popup
        if (target && target.closest(".custom-dropdown-overlay")) {
          return;
        }
        if (
          openMenu === file.id &&
          cardRef.current &&
          !cardRef.current.contains(event.target as Node)
        ) {
          setOpenMenu(null);
```

```
      }
    }

    document.addEventListener("mousedown", handleClickOutside);
    return () => {
      document.removeEventListener("mousedown",
handleClickOutside);
    };
  }, [openMenu]);

  return (
    <div
      ref={cardRef}
      className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
    >

      <button
        className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
        onClick={() =>
          setOpenMenu(openMenu === file.id ? null : file.id)
        }
      >
        <MoreVertical size={18} />
      </button>

      <p className="font-semibold">{file.file_name}</p>
```

```
        {file.description && <DescriptionDisplay html={file.description}
className="text-xs text-slate-500 dark:text-slate-400" />}

        <p className="text-xs">
          Clients: {file.clients.map(c => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map(p => p.name).join(", ") || "None"}
        </p>

        {file.asset_type === "file" && file.sharepoint_url ? (
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            View File
          </a>
        ) : file.asset_type === "url" && file.external_url ? (
          isPlayableVideoUrl(file.external_url) ? (
            <button
              onClick={() => openUrlVideo(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5
text-xs rounded-lg bg-emerald-600 text-white hover:bg-emerald-700
transition"
            >
```

```
            ▶️ Play Video
          </button>
        ) : (
          <a
            href={file.external_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            Open URL
          </a>
        )
      ) : (
        <span className="text-xs text-slate-400 dark:text-slate-500">
          No link available
        </span>
      )}

      {openMenu === file.id && (
        <div
          className="
            absolute right-3 top-10
            w-[260px]
            bg-white dark:bg-slate-900
            border border-slate-200 dark:border-slate-700
            p-3 rounded-lg shadow-lg
            space-y-2 z-50
          "
```

```
        >

          <CustomDropdown
            value={String(draft.clients?.[0] || "")}
            onChange={(selectedValue) => {
              const cid = Number(selectedValue);
              updateDraft(file.id, "clients", [cid]);
              updateDraft(file.id, "programs", []);
              fetchProgramsByClientGQL(file.id, cid);
            }}
            options={clients.map((c) => ({ id: String(c.id), label: c.name }))}
            placeholder="Select Client"
            shouldSort={true}
            searchable={clients.length > 5}
            className="w-full text-xs"
          />

          <CustomDropdown
            value={String(draft.programs?.[0] || "")}
            onChange={(selectedValue) =>
              updateDraft(file.id, "programs", [Number(selectedValue)])
            }
            options={(filteredPrograms[file.id] || []).map((p) => ({
              id: String(p.id),
              label: p.name,
            }))}
            placeholder="Select Program"
            shouldSort={true}
            className="w-full text-xs"
```

```
            searchable={(filteredPrograms[file.id] || []).length > 5}
          />

          <div className="flex gap-2 pt-1">
            <button
              onClick={() => saveAssignmentREST(file.id)}
              disabled={disableSave}
              className="flex-1 px-2 py-1 text-xs rounded bg-emerald-
600 text-white disabled:opacity-50"
            >
              Save
            </button>

            <button
              onClick={() => requestDeleteFile(file.id)}
              className="flex-1 px-2 py-1 text-xs rounded bg-red-600
text-white"
            >
              Delete
            </button>
          </div>
        </div>
      )}
    </div>
  );
}

return (
  <div className="space-y-6">
```

```
          <div className="mb-4 flex justify-center">
      <div className="relative w-full max-w-md">
        <svg
          className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
          />
        </svg>
        <input
          type="text"
          placeholder="Search files..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
        />
      </div>
    </div>
      <ConfirmDialog
```

```
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">

        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100">📂
UnAssigned Files</h3>
          <div className="space-y-3 h-[500px] overflow-y-auto">
            {unassigned.length > 0 ? (
              unassigned.map(file => (
```

```
            <FileCard key={file.id} {...file} />
          ))
        ) : (
          <div className="text-center py-8 text-slate-500 dark:text-slate-400">
            No unassigned files match your search
          </div>
        )}
      </div>
    </div>

    <div className="bg-white dark:bg-slate-900 p-4 rounded-xl border border-slate-200 dark:border-slate-700">
      <h3 className="mb-3 text-slate-900 dark:text-slate-100">✅ Assigned Files</h3>
      <div className="space-y-3 h-[500px] overflow-y-auto">
        {assigned.length > 0 ? (
          assigned.map(file => (
            <FileCard key={file.id} {...file} />
          ))
        ) : (
          <div className="text-center py-8 text-slate-500 dark:text-slate-400">
            No assigned files match your search
          </div>
        )}
      </div>
    </div>
```

```
      </div>
    </div>
  );
}
```

this is my uploadedfileslist.tsx
of feature beanch

i do created urlvideopopup.tsx in components
below is my current working code of working branch i want you to help me how to imprt urlvideopopup in below code files and make it work exactlly like above code

```
"use client";

import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
import DescriptionDisplay from "@/components/DescriptionDisplay";

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function DashboardAssets({
  clientId,
  programId,
```

```
  embedded,
}: {
  clientId: number | null;
  programId: number | null;
  /** When true, render only inner content (no outer card) for use inside
a parent card */
  embedded?: boolean;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);

  //  Load files when sidebar selection changes
  useEffect(() => {
    if (!clientId || !programId) {
      setFiles([]);
      return;
    }

    setLoading(true);

    fetch(`${API}/api/attachments/?
client=${clientId}&program=${programId}`)
      .then((r) => r.json())
      .then(setFiles)
      .finally(() => setLoading(false));
  }, [clientId, programId]);

  // If nothing selected in sidebar
```

```
  if (!clientId || !programId) {
    const fallback = (
      <p className="text-sm text-slate-500 dark:text-slate-400">
        Select client and program from sidebar to view assets.
      </p>
    );
    if (embedded) return fallback;
    return (
      <div className="p-4 border rounded bg-white dark:bg-slate-
900">
        {fallback}
      </div>
    );
  }

  // Inner item style matching Program Statistics cards: rounded-lg bg-
white border border-slate-100
  const fileItem = (f: Attachment) => (
    <div key={f.id} className="rounded-lg bg-white p-2.5 sm:p-3 text-
sm border border-slate-100 dark:bg-slate-900 dark:border-slate-800
min-w-0">
      <div className="text-xs font-medium text-slate-900 dark:text-
slate-50 break-words">{f.file_name}</div>
      {f.description && (
        <DescriptionDisplay html={f.description} className="text-xs text-
slate-500 dark:text-slate-400 mt-0.5" />
      )}
      {(f.asset_type === "file" && f.sharepoint_url) ||
      (f.asset_type === "url" && f.external_url) ? (
```

```
        <a
          href={
            f.asset_type === "file"
              ? f.sharepoint_url!
              : f.external_url!
          }
          target="_blank"
          rel="noopener noreferrer"
          className="inline-flex items-center gap-1 mt-1.5 text-xs font-
medium text-emerald-600 dark:text-emerald-400 hover:underline"
        >
          View {f.asset_type === "file" ? "file" : "link"}
          <span aria-hidden>→</span>
        </a>
      ) : null}
    </div>
  );

  const listContent = (
    <>
      {loading && (
        <div className="flex flex-col items-center justify-center py-8
text-center">
          <div className="rounded-full bg-slate-100 dark:bg-slate-800 p-
3 mb-2">
            <FolderOpen className="h-5 w-5 text-slate-400 dark:text-
slate-500 animate-pulse" aria-hidden />
          </div>
          <p className="text-sm text-slate-500 dark:text-slate-
```

```
400">Loading files...</p>
        </div>
      )}
      {files.length === 0 && !loading && (
        <div className="flex flex-col items-center justify-center py-8 px-
4 text-center rounded-lg border border-dashed border-slate-200
dark:border-slate-700 bg-slate-50/50 dark:bg-slate-800/30 min-h-
[120px]">
          <FileX2 className="h-8 w-8 text-slate-300 dark:text-slate-600
mb-2" aria-hidden />
          <p className="text-sm font-medium text-slate-600 dark:text-
slate-400">No files assigned</p>
          <p className="text-xs text-slate-500 dark:text-slate-400 mt-
0.5">Assets will appear here when assigned to this program.</p>
        </div>
      )}
      {files.length > 0 && !loading && (
        <div className="grid gap-3">
          {files.map((f) => fileItem(f))}
        </div>
      )}
    </>
  );

  if (embedded) {
    return (
      <div className="flex flex-col flex-1 min-h-0">
        <div className="flex items-center justify-between flex-shrink-0">
          <div className="inline-flex items-center gap-2 text-xs font-
```

```
medium text-slate-600 dark:text-slate-400">
            <FolderOpen className="h-4 w-4 text-emerald-500 flex-
shrink-0" aria-hidden />
            <span className="break-words">Assigned Assets
({files.length})</span>
          </div>
        </div>
        <div className="mt-4 flex-1 min-h-0 overflow-y-auto">
          {listContent}
        </div>
      </div>
    );
  }

  return (
    <section className="mt-6 rounded-xl border border-slate-200
dark:border-slate-700 bg-white dark:bg-slate-900 p-4">
      <h3 className="text-sm font-semibold mb-3 text-slate-900
dark:text-slate-50">
        Assigned Assets ({files.length})
      </h3>
      {listContent}
    </section>
  );
}
this is dashboardassets.tsx
"use client";

import { FileText, FolderOpen } from "lucide-react";
```

```
import { useEffect, useState, useCallback } from "react";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);

  const loadFiles = useCallback(async () => {
    if (!programId) {
      setFiles([]);
      return;
    }

    setLoading(true);

    try {
      const res = await fetch(`${API}/api/attachments/?
```

```
program=${programId} );
      const data = await res.json();
      setFiles(data);
    } catch (err) {
      console.error("Failed to load program files", err);
    } finally {
      setLoading(false);
    }
  }, [programId, API]);

  // Initial load + reload when program changes
  useEffect(() => {
    loadFiles();
  }, [loadFiles]);

  // Listen for attachment updates from admin dashboard
  useEffect(() => {
    function handleAttachmentsUpdated() {
      loadFiles();
    }

    function handleStorageChange(event: StorageEvent) {
      if (event.key === "attachments_updated_at") {
        loadFiles();
      }
    }

    function handleFocus() {
      loadFiles();
```

```
    }

    function handleVisibilityChange() {
      if (document.visibilityState === "visible") {
        loadFiles();
      }
    }

    window.addEventListener("attachments-updated",
handleAttachmentsUpdated);
    window.addEventListener("storage", handleStorageChange);
    window.addEventListener("focus", handleFocus);
    document.addEventListener("visibilitychange",
handleVisibilityChange);

    return () => {
      window.removeEventListener("attachments-updated",
handleAttachmentsUpdated);
      window.removeEventListener("storage", handleStorageChange);
      window.removeEventListener("focus", handleFocus);
      document.removeEventListener("visibilitychange",
handleVisibilityChange);
    };
  }, [loadFiles]);

  if (!programId) return null;

  return (
    <div className="flex flex-col">
```

```
      {/* Header (outside the box) */}
      <div className="flex items-center gap-2 mb-3">
        <div className="flex h-8 w-8 items-center justify-center
rounded-lg bg-gradient-to-br from-sky-500 to-sky-600">
          <FolderOpen className="h-4 w-4 text-white" />
        </div>
        <div>
          <h2 className="text-xs lg:text-sm font-semibold text-slate-900
dark:text-slate-50">
            Learning Resources
          </h2>
          <p className="text-[9px] lg:text-xs text-slate-600 dark:text-
slate-400">
            Files & materials
          </p>
        </div>
      </div>

      {/* Content Box */}
      <div className="h-[100px] overflow-y-auto rounded-lg border
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/70
p-2.5">
        {loading ? (
          <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
            <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-
slate-600" />
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-
```

```
slate-600 dark:text-slate-400">
            Loading files...
          </p>
        </div>
      ) : files.length === 0 ? (
        <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
          <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-
slate-600" />
          <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
            No files assigned to this program
          </p>
        </div>
      ) : (
        <div className="space-y-2">
          {files.map((file) => (
            <a
              key={file.id}
              href={
                file.asset_type === "file"
                  ? file.sharepoint_url || "#"
                  : file.external_url || "#"
              }
              target="_blank"
              rel="noopener noreferrer"
              className="flex items-center gap-2 rounded-lg border
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/60
```

```
p-2 hover:bg-slate-50 dark:hover:bg-slate-800 transition-colors"
            >
              <div className="flex h-7 w-7 flex-shrink-0 items-center
justify-center rounded-md bg-slate-100 dark:bg-slate-800 text-slate-
600 dark:text-slate-300">
                <FileText className="h-3.5 w-3.5" />
              </div>
              <span className="text-[10px] lg:text-xs font-medium text-
emerald-600 truncate">
                {file.file_name}
              </span>
            </a>
          ))}
        </div>
      )}
    </div>
  </div>
 );
}
```

this is programattachments.tsx

```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import DescriptionDisplay from "@/components/DescriptionDisplay";

type Option = { id: number; name: string };
```

```
type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: Option[];
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const { toasts, showToast, removeToast } = useToast();
```

```
  function notifyAttachmentsUpdated() {
    localStorage.setItem("attachments_updated_at",
Date.now().toString());
  }

  async function fetchFilesREST() {
    try {
      // Always fetch ALL files.
      // Left side and right side are split in frontend based on selected
sidebar client/program.
      const res = await fetch(`${API}/api/attachments/`);
      const data = await res.json();
      setFiles(data);
    } catch (err) {
      console.error("Failed to fetch attachments:", err);
      showToast("Failed to load files", "error");
    }
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey]);

  async function assignFileToCurrentSelection(file: Attachment) {
    if (!clientId || !programId) {
      showToast("Please select a Client and Program from the sidebar
first", "error");
      return;
```

```
    }

    try {
      const res = await fetch(`${API}/api/attachments/${file.id}/`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          clients: [clientId],
          programs: [programId],
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(data?.error || "Failed to assign file");
      }

      // Optimistic UI update:
      // Move file to current selected client/program immediately
      setFiles((prev) =>
        prev.map((f) =>
          f.id === file.id
            ? {
                ...f,
                clients: Array.isArray(data?.clients) && data.clients.length > 0
                  ? data.clients
                  : [{ id: clientId, name: "Selected Client" }],
                programs: Array.isArray(data?.programs) &&
```

```
data.programs.length > 0
                ? data.programs
                : [{ id: programId, name: "Selected Program" }],
            }
          : f
      )
    );

    setOpenMenu(null);
    showToast("File assigned successfully", "success");

    // Optional if you want server truth refresh:
    // await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Assignment failed", "error");
  }
}

async function unassignFile(fileId: number) {
  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        clients: [],
        programs: [],
      }),
    });
```

```
      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(data?.error || "Failed to unassign file");
      }

      // Optimistic UI update:
      // Move file from right side -> left side immediately
      setFiles((prev) =>
        prev.map((f) =>
          f.id === fileId
            ? {
                ...f,
                clients: [],
                programs: [],
              }
            : f
        )
      );

      setOpenMenu(null);
      showToast("File unassigned successfully", "success");

      // Optional if you want server truth refresh:
      // await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Unassign failed", "error");
    }
  }
```

```
function requestDeleteFile(fileId: number) {
  setOpenMenu(null);
  setFileIdToDelete(fileId);
}

async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;

  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles((prev) => prev.filter((f) => f.id !== fileId));
    showToast("File deleted successfully", "success");
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}
```

```
  const filteredFiles = files.filter((f) =>
    f.file_name.toLowerCase().includes(search.toLowerCase())
  );

  // Right side:
  // Files assigned to CURRENT selected sidebar client + program
  const assigned = filteredFiles.filter((f) => {
    if (!clientId || !programId) return false;

    const matchesClient = f.clients.some((c) => c.id === clientId);
    const matchesProgram = f.programs.some((p) => p.id ===
programId);

    return matchesClient && matchesProgram;
  });

  // Left side:
  // Files NOT assigned to CURRENT selected sidebar client + program
  // Includes:
  // 1. truly unassigned files
  // 2. files assigned to some other client/program
  const available = filteredFiles.filter((f) => {
    if (!clientId || !programId) {
      // If sidebar selection is incomplete, show only truly global
unassigned
      return f.clients.length === 0 && f.programs.length === 0;
    }
```

```
    const matchesClient = f.clients.some((c) => c.id === clientId);
    const matchesProgram = f.programs.some((p) => p.id ===
programId);

    // If already assigned to current selection, exclude from left side
    return !(matchesClient && matchesProgram);
  });

  function FileCard({
    file,
    isAvailableSection,
  }: {
    file: Attachment;
    isAvailableSection: boolean;
  }) {
    const cardRef = useRef<HTMLDivElement>(null);

    const isTrulyUnassigned =
      file.clients.length === 0 && file.programs.length === 0;

    useEffect(() => {
      function handleClickOutside(event: MouseEvent) {
        if (
          openMenu === file.id &&
          cardRef.current &&
          !cardRef.current.contains(event.target as Node)
        ) {
          setOpenMenu(null);
        }
```

```
      }

      document.addEventListener("mousedown", handleClickOutside);
      return () => {
        document.removeEventListener("mousedown",
handleClickOutside);
      };
    }, [openMenu, file.id]);

    return (
      <div
        ref={cardRef}
        className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
      >
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
          onClick={() => setOpenMenu(openMenu === file.id ? null :
file.id)}
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold pr-8 break-words">{file.file_name}
</p>

        {file.description && (
```

```
          <DescriptionDisplay
            html={file.description}
            className="text-xs text-slate-500 dark:text-slate-400"
          />
        )}

        <p className="text-xs">
          Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
        </p>

        {(file.asset_type === "file" && file.sharepoint_url) ||
        (file.asset_type === "url" && file.external_url) ? (
          <a
            href={
              file.asset_type === "file"
                ? file.sharepoint_url!
                : file.external_url!
            }
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            View {file.asset_type === "file" ? "File" : "Link"}
          </a>
```

```
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}

        {openMenu === file.id && (
          <div
            className="
              absolute right-3 top-10
              w-[180px]
              bg-white dark:bg-slate-900
              border border-slate-200 dark:border-slate-700
              p-3 rounded-lg shadow-lg
              space-y-2 z-50
            "
          >
            {isAvailableSection ? (
              <>
                <button
                  onClick={() => assignFileToCurrentSelection(file)}
                  className="w-full px-3 py-2 text-sm rounded bg-emerald-
600 text-white hover:bg-emerald-700"
                >
                  Assign
                </button>

                {/* Delete only if file is truly global unassigned */}
                {isTrulyUnassigned && (
```

```
                <button
                  onClick={() => requestDeleteFile(file.id)}
                  className="w-full px-3 py-2 text-sm rounded bg-red-600
text-white hover:bg-red-700"
                >
                  Delete
                </button>
              )}
            </>
          ) : (
            <button
              onClick={() => unassignFile(file.id)}
              className="w-full px-3 py-2 text-sm rounded bg-amber-500
text-white hover:bg-amber-600"
            >
              Unassign
            </button>
          )}
        </div>
      )}
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="mb-4 flex justify-center">
      <div className="relative w-full max-w-md">
        <svg
```

```
          className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
         >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
          />
         </svg>

         <input
          type="text"
          placeholder="Search files..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
         />
        </div>
      </div>

      <ConfirmDialog
       open={fileIdToDelete != null}
```

```
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Available Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100">
            📂 Available Files
          </h3>

          <div className="space-y-3 h-[500px] overflow-y-auto">
            {available.length > 0 ? (
              available.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  isAvailableSection={true}
                />
              ))
            ) : (
```

```
            <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
              No available files match your search
            </div>
          )}
        </div>
      </div>

      {/* Assigned Files */}
      <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
        <h3 className="mb-3 text-slate-900 dark:text-slate-100">
          ✅ Assigned Files
        </h3>

        <div className="space-y-3 h-[500px] overflow-y-auto">
          {assigned.length > 0 ? (
            assigned.map((file) => (
              <FileCard
                key={file.id}
                file={file}
                isAvailableSection={false}
              />
            ))
          ) : (
            <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
              {!clientId || !programId
                ? "Select a client and program from the sidebar to view
```

```
assigned files"
                : "No assigned files match your search"}
            </div>
          )}
        </div>
      </div>
    </div>
  </div>
 );
}
```

this is uploadedfileslist.tsx
now tell me accordinglly how to do it step by step in detail

Perfect — you already have the **working implementation in your feature branch**, so the easiest + safest way is to **copy the same video popup logic into your current working branch files**.

I'll show you **exactly what to change in each file** so it behaves **exactly like your feature branch**.

✅ Goal

You want these 3 files in your **working branch** to support:

- **URL videos** (YouTube / Vimeo / Loom / direct `.mp4`, etc.)
- Show ▶️ **Play video** button instead of normal link
- Open in your `UrlVideoPopup` component
- Non-video URLs should still open as normal link

Files to update:

1. `dashboardassets.tsx`
2. `programattachments.tsx`
3. `uploadedfileslist.tsx`

✅ Step 1: Make sure `UrlVideoPopup.tsx` exists

You already said you created it in `components`.

So just ensure file exists here:

```bash
```

or if your alias points directly:

```bash
```

Since your imports use:

```typescript
```

that path should already be correct.

✅ Step 2: Update `dashboardassets.tsx`

A. Add import

Existing imports:

```
import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
```

TypeScript

Change to:

```
import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
import DescriptionDisplay from "@/components/DescriptionDisplay";
```

TypeScript

B. Add popup state

Existing state:

TypeScript

Change to:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [loading, setLoading] = useState(false);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

C. Add helper functions

Add these **below your** `if (!clientId || !programId)` **block** (same place as in feature branch):

```
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
```

```
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part === "share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
```

```
      }

      const embedIndex = pathParts.findIndex((part) => part === "embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) return;

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
```

```
  });
}
```

D. Replace link rendering inside `fileItem`

Current code inside `fileItem`:

```
{(f.asset_type === "file" && f.sharepoint_url) ||
(f.asset_type === "url" && f.external_url) ? (
  <a
    href={
      f.asset_type === "file"
        ? f.sharepoint_url!
        : f.external_url!
    }
    target="_blank"
    rel="noopener noreferrer"
    className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 dark:te
  >
    View {f.asset_type === "file" ? "file" : "link"}
    <span aria-hidden>→</span>
  </a>
) : null}
```

Replace with:

```
{f.asset_type === "file" && f.sharepoint_url ? (
  <a
    href={f.sharepoint_url}
    target="_blank"
    rel="noopener noreferrer"
    className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 dark:te
  >
    View file
    <span aria-hidden>→</span>
  </a>
) : f.asset_type === "url" && f.external_url ? (
  isPlayableVideoUrl(f.external_url) ? (
    <button
      onClick={() => openUrlVideo(f)}
      className="inline-flex items-center gap-1 mt-1.5 px-2.5 py-1 text-xs font-medium rounded-lg
    >
      ▶ Play video
    </button>
  ) : (
    <a
      href={f.external_url}
      target="_blank"
      rel="noopener noreferrer"
      className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 dark:
    >
      Open link
      <span aria-hidden>→</span>
    </a>
  )
) : null}
```

E. Add `UrlVideoPopup` in embedded return

Current embedded return:

```
if (embedded) {
  return (
    <div className="flex flex-col flex-1 min-h-0">
      ...
    </div>
  );
}
```

Replace with:

```
if (embedded) {
  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="flex flex-col flex-1 min-h-0">
        <div className="flex items-center justify-between flex-shrink-0">
          <div className="inline-flex items-center gap-2 text-xs font-medium text-slate-600 dark:
            <FolderOpen className="h-4 w-4 text-emerald-500 flex-shrink-0" aria-hidden />
            <span className="break-words">Assigned Assets ({files.length})</span>
          </div>
```

```
          </div>
          <div className="mt-4 flex-1 min-h-0 overflow-y-auto">
            {listContent}
          </div>
        </div>
      </>
    );
  }
```

F. Add `UrlVideoPopup` in normal return

Current return:

```
return (
  <section className="mt-6 rounded-xl border border-slate-200 dark:border-slate-700 bg-white dark
    ...
  </section>
);
```

Replace with:

```
return (
  <>
    <UrlVideoPopup
      open={videoPopup != null}
```

```
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <section className="mt-6 rounded-xl border border-slate-200 dark:border-slate-700 bg-white da
        <h3 className="text-sm font-semibold mb-3 text-slate-900 dark:text-slate-50">
          Assigned Assets ({files.length})
        </h3>
        {listContent}
      </section>
    </>
  );
```

✅ Step 3: Update `programattachments.tsx`

This one is straightforward.

A. Add import

Existing:

```
import { FileText, FolderOpen } from "lucide-react";
```

‹/› TypeScript

Change to:

```
import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState, useCallback } from "react";
```

TypeScript

B. Add popup state

Existing:

```
const [files, setFiles] = useState<Attachment[]>([]);
```

TypeScript

Change to:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [loading, setLoading] = useState(false);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

C. Add helper functions

Add below the `useEffect` blocks, before `if (!programId) return null;`

```
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      // already embed URL
```

```
      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part === "share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
      }

      const embedIndex = pathParts.findIndex((part) => part === "embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}
```

```
function openUrlVideo(file: Attachment) {
  if (!file.external_url) return;

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}
```

D. Add popup component in return

Inside the main return, at the top of the fragment, add:

```
<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
```

TypeScript

So your return should start like:

```
return (
  <>
    <UrlVideoPopup
      open={videoPopup != null}
      title={videoPopup?.title || "Video Player"}
      videoUrl={videoPopup?.videoUrl || null}
      embedUrl={videoPopup?.embedUrl || null}
      isDirectVideo={videoPopup?.isDirectVideo || false}
      onClose={() => setVideoPopup(null)}
    />

    <div className="flex flex-col">
      ...
    </div>
  </>
);
```

E. Replace file list item rendering

Current map block:

```
{files.map((file) => (
  <a
    key={file.id}
    href={
      file.asset_type === "file"
        ? file.sharepoint_url || "#"
        : file.external_url || "#"
    }
    target="_blank"
    rel="noopener noreferrer"
    className="flex items-center gap-2 rounded-lg border border-slate-200 dark:border-slate-800 b
  >
    <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounded-md bg-slate-10
      <FileText className="h-3.5 w-3.5" />
    </div>
    <span className="text-[10px] lg:text-xs font-medium text-emerald-600 truncate">
      {file.file_name}
    </span>
  </a>
))}
```

Replace with:

```
{files.map((file) => {
  const isVideoUrl =
    file.asset_type === "url" &&
    !!file.external_url &&
    isPlayableVideoUrl(file.external_url);

  return (
    <div
```

```
        key={file.id}
        className="rounded-lg border border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-
      >
        <div className="flex items-start gap-2">
          <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center rounded-md bg-slat
            <FileText className="h-3.5 w-3.5" />
          </div>

          <div className="min-w-0 flex-1">
            <span className="block text-[10px] lg:text-xs font-medium text-slate-900 dark:text-slat
              {file.file_name}
            </span>

            <div className="mt-1.5">
              {file.asset_type === "file" && file.sharepoint_url ? (
                <a
                  href={file.sharepoint_url}
                  target="_blank"
                  rel="noopener noreferrer"
                  className="inline-flex items-center gap-1 text-[10px] lg:text-xs font-medium text
                >
                  View file
                  <span aria-hidden>→</span>
                </a>
              ) : file.asset_type === "url" && file.external_url ? (
                isVideoUrl ? (
                  <button
                    type="button"
                    onClick={() => openUrlVideo(file)}
                    className="inline-flex items-center gap-1 px-2.5 py-1 text-[10px] lg:text-xs fo
                  >
                    ▶ Play video
                  </button>
                ) : (
                  <a
```

```
                    href={file.external_url}
                    target="_blank"
                    rel="noopener noreferrer"
                    className="inline-flex items-center gap-1 text-[10px] lg:text-xs font-medium te
                  >
                    Open link
                    <span aria-hidden>→</span>
                  </a>
                )
              ) : (
                <span className="text-[10px] lg:text-xs text-slate-400 dark:text-slate-500">
                  No link available
                </span>
              )}
            </div>
          </div>
        </div>
      </div>
    );
  })}
```

✅ Step 4: Update `uploadedfileslist.tsx`

This one is also easy because your feature branch already has almost exact logic.

A. Add import

Existing:

TypeScript

Change to:

```
import DescriptionDisplay from "@/components/DescriptionDisplay";
```

TypeScript

B. Add popup state

Existing state:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [openMenu, setOpenMenu] = useState<number | null>(null);
const [search, setSearch] = useState("");
```

TypeScript

Change to:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [openMenu, setOpenMenu] = useState<number | null>(null);
const [search, setSearch] = useState("");
```

```
const [fileIdToDelete, setFileIdToDelete] = useState<number | null>(null);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

C. Add helper functions

Add these **below** `confirmDeleteFile()` (or before `filteredFiles`):

```
function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link: youtu.be/VIDEO_ID
    if (host.includes("youtu.be")) {
```

```
    const videoId = parsed.pathname.replace("/", "");
    return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
  }

  // YouTube normal link: youtube.com/watch?v=VIDEO_ID
  if (host.includes("youtube.com")) {
    const videoId = parsed.searchParams.get("v");
    if (videoId) return `https://www.youtube.com/embed/${videoId}`;

    // already embed url
    if (parsed.pathname.includes("/embed/")) return url;
  }

  // Vimeo
  if (host.includes("vimeo.com")) {
    const pathParts = parsed.pathname.split("/").filter(Boolean);
    const videoId = pathParts[pathParts.length - 1];
    return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
  }

  // Loom
  if (host.includes("loom.com")) {
    const pathParts = parsed.pathname.split("/").filter(Boolean);
    const shareIndex = pathParts.findIndex((part) => part === "share");
    if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
      return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
    }

    const embedIndex = pathParts.findIndex((part) => part === "embed");
    if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
      return url;
    }
  }

  return null;
```

```
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}
```

D. Add popup component in main return

After `ToastContainer`, add:

```
<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
  onClose={() => setVideoPopup(null)}
/>
```

So this section becomes:

```
<ToastContainer toasts={toasts} onClose={removeToast} />

<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
  onClose={() => setVideoPopup(null)}
/>
```

E. Replace link rendering inside `FileCard`

Current code:

```
{(file.asset_type === "file" && file.sharepoint_url) ||
(file.asset_type === "url" && file.external_url) ? (
  <a
    href={
      file.asset_type === "file"
        ? file.sharepoint_url!
        : file.external_url!
    }
    target="_blank"
    rel="noopener noreferrer"
    className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark:hover:t
  >
    View {file.asset_type === "file" ? "File" : "Link"}
  </a>
) : (
  <span className="text-xs text-slate-400 dark:text-slate-500">
    No link available
  </span>
)}
```

Replace with:

```
{file.asset_type === "file" && file.sharepoint_url ? (
  <a
    href={file.sharepoint_url}
    target="_blank"
    rel="noopener noreferrer"
    className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark:hover:t
  >
```

```
    View File
  </a>
) : file.asset_type === "url" && file.external_url ? (
  isPlayableVideoUrl(file.external_url) ? (
    <button
      onClick={() => openUrlVideo(file)}
      className="mt-2 inline-flex items-center gap-2 px-3 py-1.5 text-xs rounded-lg bg-emerald-60
    >
      ▶ Play Video
    </button>
  ) : (
    <a
      href={file.external_url}
      target="_blank"
      rel="noopener noreferrer"
      className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark:hover
    >
      Open URL
    </a>
  )
) : (
  <span className="text-xs text-slate-400 dark:text-slate-500">
    No link available
  </span>
)}
```

✅ Step 5: Important small fix in `dashboardassets.tsx`

In your current working branch `useEffect`, dependency array is:

TypeScript

Since you use `API`, better make it:

TypeScript

This avoids lint warnings and keeps consistency.

✅ Step 6: Test cases you should verify

After changes, test these:

1. SharePoint file

- `asset_type = "file"`
- `sharepoint_url = ...`
- Should show **View file**
- Opens in new tab

2. Normal URL (non-video)

Example:

- `https://google.com`
- Should show **Open link** / **Open URL**
- Opens in new tab

3. YouTube URL

Example:

- `https://www.youtube.com/watch?v=abc123`
- Should show ▶️ **Play video**
- Opens popup

4. YouTube short URL

Example:

- `https://youtu.be/abc123`
- Should show ▶️ **Play video**

5. Vimeo URL

Example:

- `https://vimeo.com/123456`

6. Loom share URL

Example:

- `https://www.loom.com/share/abcd1234`

7. Direct MP4

Example:

- `https://example.com/video.mp4`

✅ Step 7: Best approach (recommended)

Since your feature branch code is already correct, the **fastest and safest approach** is:

- For `dashboardassets.tsx` → copy the feature branch version almost fully
- For `programattachments.tsx` → copy the feature branch version almost fully
- For `uploadedfileslist.tsx` → copy only the **video popup parts**, because your working branch has different assignment logic

This is the best path because your working branch `uploadedfileslist.tsx` has custom behavior:

- available files vs assigned files based on selected sidebar
- assign/unassign to current selection
- delete only truly unassigned

So **don't replace the whole file** there — only add:

- import
- state
- helper functions
- popup component
- link/button rendering change

✅ Super important note (common issue)

If `UrlVideoPopup` uses a modal with portal / overlay, make sure:

- it has proper `z-index` (`z-50` or higher)
- parent containers do **not** block it with `overflow-hidden`

If popup opens but is not visible, this is usually the reason.

✅ If you want the fastest possible way

If you want, I can do the **best next step** for you right now:

I can give you all 3 final full updated files (ready to paste)

- `dashboardassets.tsx`
- `programattachments.tsx`
- `uploadedfileslist.tsx`

👉 That will be much easier than manual step-by-step editing and will avoid syntax mistakes.

If you want, reply:

"give me final full updated code for all 3 files"

and I'll paste all 3 complete files exactly in your current working-branch structure.

can you give me fully updated codefiles for programattachments, dashboardassets and uploadedfileslist

Yes — **best approach now is full ready-to-paste files** 👍
I'll give you **fully updated code** for all 3 files, using your **current working branch logic**, but with `UrlVideoPopup` **integrated exactly like your feature branch**.

✅ 1) `dashboardassets.tsx` (fully updated)

```
"use client";

import { useEffect, useState } from "react";
import { FolderOpen, FileX2 } from "lucide-react";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function DashboardAssets({
  clientId,
  programId,
  embedded,
}: {
  clientId: number | null;
  programId: number | null;
  /** When true, render only inner content (no outer card) for use inside a parent card */
  embedded?: boolean;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
```

```
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  // Load files when sidebar selection changes
  useEffect(() => {
    if (!clientId || !programId) {
      setFiles([]);
      return;
    }

    setLoading(true);

    fetch(`${API}/api/attachments/?client=${clientId}&program=${programId}`)
      .then((r) => r.json())
      .then(setFiles)
      .finally(() => setLoading(false));
  }, [clientId, programId, API]);

  // If nothing selected in sidebar
  if (!clientId || !programId) {
    const fallback = (
      <p className="text-sm text-slate-500 dark:text-slate-400">
        Select client and program from sidebar to view assets.
      </p>
    );

    if (embedded) return fallback;

    return (
      <div className="p-4 border rounded bg-white dark:bg-slate-900">
        {fallback}
```

```
      </div>
    );
  }

  function isDirectVideoUrl(url: string) {
    const lower = url.toLowerCase();
    return (
      lower.endsWith(".mp4") ||
      lower.endsWith(".webm") ||
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();

      // YouTube short link
      if (host.includes("youtu.be")) {
        const videoId = parsed.pathname.replace("/", "");
        return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
      }

      // YouTube normal link
      if (host.includes("youtube.com")) {
        const videoId = parsed.searchParams.get("v");
        if (videoId) return `https://www.youtube.com/embed/${videoId}`;

        if (parsed.pathname.includes("/embed/")) return url;
      }

      // Vimeo
```

```
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
      }

      // Loom
      if (host.includes("loom.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);

        const shareIndex = pathParts.findIndex((part) => part === "share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
        }

        const embedIndex = pathParts.findIndex((part) => part === "embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
  }

  function openUrlVideo(file: Attachment) {
    if (!file.external_url) return;

    const directVideo = isDirectVideoUrl(file.external_url);
```

```
    const embedUrl = getEmbedUrl(file.external_url);

    if (!directVideo && !embedUrl) {
      window.open(file.external_url, "_blank", "noopener,noreferrer");
      return;
    }

    setVideoPopup({
      title: file.file_name,
      videoUrl: file.external_url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }

  // Inner item style matching Program Statistics cards
  const fileItem = (f: Attachment) => (
    <div
      key={f.id}
      className="rounded-lg bg-white p-2.5 sm:p-3 text-sm border border-slate-100 dark:bg-slate-9
    >
      <div className="text-xs font-medium text-slate-900 dark:text-slate-50 break-words">
        {f.file_name}
      </div>

      {f.description && (
        <DescriptionDisplay
          html={f.description}
          className="text-xs text-slate-500 dark:text-slate-400 mt-0.5"
        />
      )}

      {f.asset_type === "file" && f.sharepoint_url ? (
        <a
          href={f.sharepoint_url}
```

```
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600 d
          >
            View file
            <span aria-hidden>→</span>
          </a>
        ) : f.asset_type === "url" && f.external_url ? (
          isPlayableVideoUrl(f.external_url) ? (
            <button
              onClick={() => openUrlVideo(f)}
              className="inline-flex items-center gap-1 mt-1.5 px-2.5 py-1 text-xs font-medium roun
            >
              ▶ Play video
            </button>
          ) : (
            <a
              href={f.external_url}
              target="_blank"
              rel="noopener noreferrer"
              className="inline-flex items-center gap-1 mt-1.5 text-xs font-medium text-emerald-600
            >
              Open link
              <span aria-hidden>→</span>
            </a>
          )
        ) : null}
      </div>
    );

    const listContent = (
      <>
        {loading && (
          <div className="flex flex-col items-center justify-center py-8 text-center">
            <div className="rounded-full bg-slate-100 dark:bg-slate-800 p-3 mb-2">
```

```
            <FolderOpen
              className="h-5 w-5 text-slate-400 dark:text-slate-500 animate-pulse"
              aria-hidden
            />
          </div>
          <p className="text-sm text-slate-500 dark:text-slate-400">
            Loading files…
          </p>
        </div>
      )}

      {files.length === 0 && !loading && (
        <div className="flex flex-col items-center justify-center py-8 px-4 text-center rounded-l
          <FileX2
            className="h-8 w-8 text-slate-300 dark:text-slate-600 mb-2"
            aria-hidden
          />
          <p className="text-sm font-medium text-slate-600 dark:text-slate-400">
            No files assigned
          </p>
          <p className="text-xs text-slate-500 dark:text-slate-400 mt-0.5">
            Assets will appear here when assigned to this program.
          </p>
        </div>
      )}

      {files.length > 0 && !loading && (
        <div className="grid gap-3">{files.map((f) => fileItem(f))}</div>
      )}
    </>
  );

  if (embedded) {
    return (
      <>
```

```
        <UrlVideoPopup
          open={videoPopup != null}
          title={videoPopup?.title || "Video Player"}
          videoUrl={videoPopup?.videoUrl || null}
          embedUrl={videoPopup?.embedUrl || null}
          isDirectVideo={videoPopup?.isDirectVideo || false}
          onClose={() => setVideoPopup(null)}
        />

        <div className="flex flex-col flex-1 min-h-0">
          <div className="flex items-center justify-between flex-shrink-0">
            <div className="inline-flex items-center gap-2 text-xs font-medium text-slate-600 dar
              <FolderOpen
                className="h-4 w-4 text-emerald-500 flex-shrink-0"
                aria-hidden
              />
              <span className="break-words">Assigned Assets ({files.length})</span>
            </div>
          </div>

          <div className="mt-4 flex-1 min-h-0 overflow-y-auto">{listContent}</div>
        </div>
      </>
    );
  }

  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
```

```
      />

      <section className="mt-6 rounded-xl border border-slate-200 dark:border-slate-700 bg-white
        <h3 className="text-sm font-semibold mb-3 text-slate-900 dark:text-slate-50">
          Assigned Assets ({files.length})
        </h3>
        {listContent}
      </section>
    </>
  );
}
```

✅ 2) `programattachments.tsx` (fully updated)

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState, useCallback } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
```

```
    programId,
  }: {
    programId?: string | number | null;
  }) {
    const API = process.env.NEXT_PUBLIC_API_BASE_URL;

    const [files, setFiles] = useState<Attachment[]>([]);
    const [loading, setLoading] = useState(false);
    const [videoPopup, setVideoPopup] = useState<{
      title: string;
      videoUrl: string;
      embedUrl: string | null;
      isDirectVideo: boolean;
    } | null>(null);

    const loadFiles = useCallback(async () => {
      if (!programId) {
        setFiles([]);
        return;
      }

      setLoading(true);

      try {
        const res = await fetch(`${API}/api/attachments/?program=${programId}`);
        const data = await res.json();
        setFiles(data);
      } catch (err) {
        console.error("Failed to load program files", err);
        setFiles([]);
      } finally {
        setLoading(false);
      }
    }, [programId, API]);
```

```
  // Initial load + reload when program changes
  useEffect(() => {
    loadFiles();
  }, [loadFiles]);

  // Listen for attachment updates from admin dashboard
  useEffect(() => {
    function handleAttachmentsUpdated() {
      loadFiles();
    }

    function handleStorageChange(event: StorageEvent) {
      if (event.key === "attachments_updated_at") {
        loadFiles();
      }
    }

    function handleFocus() {
      loadFiles();
    }

    function handleVisibilityChange() {
      if (document.visibilityState === "visible") {
        loadFiles();
      }
    }

    window.addEventListener("attachments-updated", handleAttachmentsUpdated);
    window.addEventListener("storage", handleStorageChange);
    window.addEventListener("focus", handleFocus);
    document.addEventListener("visibilitychange", handleVisibilityChange);

    return () => {
      window.removeEventListener("attachments-updated", handleAttachmentsUpdated);
      window.removeEventListener("storage", handleStorageChange);
```

```
      window.removeEventListener("focus", handleFocus);
      document.removeEventListener("visibilitychange", handleVisibilityChange);
    };
  }, [loadFiles]);

  function isDirectVideoUrl(url: string) {
    const lower = url.toLowerCase();
    return (
      lower.endsWith(".mp4") ||
      lower.endsWith(".webm") ||
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();

      // YouTube short link
      if (host.includes("youtu.be")) {
        const videoId = parsed.pathname.replace("/", "");
        return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
      }

      // YouTube normal link
      if (host.includes("youtube.com")) {
        const videoId = parsed.searchParams.get("v");
        if (videoId) return `https://www.youtube.com/embed/${videoId}`;

        // already embed URL
        if (parsed.pathname.includes("/embed/")) return url;
      }
```

```
    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part === "share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
      }

      const embedIndex = pathParts.findIndex((part) => part === "embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) return;
```

```
    const directVideo = isDirectVideoUrl(file.external_url);
    const embedUrl = getEmbedUrl(file.external_url);

    if (!directVideo && !embedUrl) {
      window.open(file.external_url, "_blank", "noopener,noreferrer");
      return;
    }

    setVideoPopup({
      title: file.file_name,
      videoUrl: file.external_url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }

  if (!programId) return null;

  return (
    <>
      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="flex flex-col">
        {/* Header */}
        <div className="flex items-center gap-2 mb-3">
          <div className="flex h-8 w-8 items-center justify-center rounded-lg bg-gradient-to-br f
            <FolderOpen className="h-4 w-4 text-white" />
```

```
            </div>
            <div>
              <h2 className="text-xs lg:text-sm font-semibold text-slate-900 dark:text-slate-50">
                Learning Resources
              </h2>
              <p className="text-[9px] lg:text-xs text-slate-600 dark:text-slate-400">
                Files & materials
              </p>
            </div>
          </div>

          {/* Content Box */}
          <div className="h-[100px] overflow-y-auto rounded-lg border border-slate-200 dark:border-
            {loading ? (
              <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
                <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
                <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                  Loading files...
                </p>
              </div>
            ) : files.length === 0 ? (
              <div className="rounded-lg border border-dashed border-slate-300 dark:border-slate-70
                <FileText className="mx-auto h-5 w-5 text-slate-400 dark:text-slate-600" />
                <p className="mt-2 text-[10px] lg:text-xs font-medium text-slate-600 dark:text-slat
                  No files assigned to this program
                </p>
              </div>
            ) : (
              <div className="space-y-2">
                {files.map((file) => {
                  const isVideoUrl =
                    file.asset_type === "url" &&
                    !!file.external_url &&
                    isPlayableVideoUrl(file.external_url);
```

```
                return (
                  <div
                    key={file.id}
                    className="rounded-lg border border-slate-200 dark:border-slate-800 bg-white
                  >
                    <div className="flex items-start gap-2">
                      <div className="flex h-7 w-7 flex-shrink-0 items-center justify-center roun
                        <FileText className="h-3.5 w-3.5" />
                      </div>

                      <div className="min-w-0 flex-1">
                        <span className="block text-[10px] lg:text-xs font-medium text-slate-900
                          {file.file_name}
                        </span>

                        {/* Action */}
                        <div className="mt-1.5">
                          {file.asset_type === "file" && file.sharepoint_url ? (
                            <a
                              href={file.sharepoint_url}
                              target="_blank"
                              rel="noopener noreferrer"
                              className="inline-flex items-center gap-1 text-[10px] lg:text-xs fo
                            >
                              View file
                              <span aria-hidden>→</span>
                            </a>
                          ) : file.asset_type === "url" && file.external_url ? (
                            isVideoUrl ? (
                              <button
                                type="button"
                                onClick={() => openUrlVideo(file)}
                                className="inline-flex items-center gap-1 px-2.5 py-1 text-[10px]
                              >
                                ▶ Play video
```

```
                              </button>
                            ) : (
                              <a
                                href={file.external_url}
                                target="_blank"
                                rel="noopener noreferrer"
                                className="inline-flex items-center gap-1 text-[10px] lg:text-xs
                              >
                                Open link
                                <span aria-hidden>→</span>
                              </a>
                            )
                          ) : (
                            <span className="text-[10px] lg:text-xs text-slate-400 dark:text-slat
                              No link available
                            </span>
                          )}
                        </div>
                      </div>
                    </div>
                  </div>
                );
              })}
            </div>
          )}
        </div>
      </div>
    </>
  );
}
```

✅ 3) `uploadedfileslist.tsx` (fully updated)

Important: This keeps your **current working branch logic**:

- Available Files / Assigned Files split
- Assign to current sidebar selection
- Unassign
- Delete only truly unassigned
- Plus now adds **UrlVideoPopup**

```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Option = { id: number; name: string };

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: Option[];
};
```

```
export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>(null);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);
  const { toasts, showToast, removeToast } = useToast();

  function notifyAttachmentsUpdated() {
    localStorage.setItem("attachments_updated_at", Date.now().toString());
    window.dispatchEvent(new Event("attachments-updated"));
  }

  async function fetchFilesREST() {
    try {
      // Always fetch ALL files.
      // Left side and right side are split in frontend based on selected sidebar client/program.
      const res = await fetch(`${API}/api/attachments/`);
      const data = await res.json();
      setFiles(data);
```

```
    } catch (err) {
      console.error("Failed to fetch attachments:", err);
      showToast("Failed to load files", "error");
    }
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey]);

  async function assignFileToCurrentSelection(file: Attachment) {
    if (!clientId || !programId) {
      showToast("Please select a Client and Program from the sidebar first", "error");
      return;
    }

    try {
      const res = await fetch(`${API}/api/attachments/${file.id}/`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          clients: [clientId],
          programs: [programId],
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(data?.error || "Failed to assign file");
      }

      // Optimistic UI update:
      // Move file to current selected client/program immediately
      setFiles((prev) =>
```

```
        prev.map((f) =>
          f.id === file.id
            ? {
                ...f,
                clients:
                  Array.isArray(data?.clients) && data.clients.length > 0
                    ? data.clients
                    : [{ id: clientId, name: "Selected Client" }],
                programs:
                  Array.isArray(data?.programs) && data.programs.length > 0
                    ? data.programs
                    : [{ id: programId, name: "Selected Program" }],
              }
            : f
        )
      );

      setOpenMenu(null);
      showToast("File assigned successfully", "success");
      notifyAttachmentsUpdated();

      // Optional if you want server truth refresh:
      // await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Assignment failed", "error");
    }
  }

  async function unassignFile(fileId: number) {
    try {
      const res = await fetch(`${API}/api/attachments/${fileId}/`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          clients: [],
```

```
          programs: [],
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(data?.error || "Failed to unassign file");
      }

      // Optimistic UI update:
      // Move file from right side -> left side immediately
      setFiles((prev) =>
        prev.map((f) =>
          f.id === fileId
            ? {
                ...f,
                clients: [],
                programs: [],
              }
            : f
        )
      );

      setOpenMenu(null);
      showToast("File unassigned successfully", "success");
      notifyAttachmentsUpdated();

      // Optional if you want server truth refresh:
      // await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Unassign failed", "error");
    }
  }
```

```
function requestDeleteFile(fileId: number) {
  setOpenMenu(null);
  setFileIdToDelete(fileId);
}

async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;

  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles((prev) => prev.filter((f) => f.id !== fileId));
    showToast("File deleted successfully", "success");
    notifyAttachmentsUpdated();
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}

function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
```

```
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();

      // YouTube short link: youtu.be/VIDEO_ID
      if (host.includes("youtu.be")) {
        const videoId = parsed.pathname.replace("/", "");
        return videoId ? `https://www.youtube.com/embed/${videoId}` : null;
      }

      // YouTube normal link: youtube.com/watch?v=VIDEO_ID
      if (host.includes("youtube.com")) {
        const videoId = parsed.searchParams.get("v");
        if (videoId) return `https://www.youtube.com/embed/${videoId}`;

        // already embed url
        if (parsed.pathname.includes("/embed/")) return url;
      }

      // Vimeo
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` : null;
      }

      // Loom
      if (host.includes("loom.com")) {
```

```
        const pathParts = parsed.pathname.split("/").filter(Boolean);

        const shareIndex = pathParts.findIndex((part) => part === "share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return `https://www.loom.com/embed/${pathParts[shareIndex + 1]}`;
        }

        const embedIndex = pathParts.findIndex((part) => part === "embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
  }

  function openUrlVideo(file: Attachment) {
    if (!file.external_url) {
      showToast("URL not available", "error");
      return;
    }

    const directVideo = isDirectVideoUrl(file.external_url);
    const embedUrl = getEmbedUrl(file.external_url);

    if (!directVideo && !embedUrl) {
      window.open(file.external_url, "_blank", "noopener,noreferrer");
      return;
```

```
    }

    setVideoPopup({
      title: file.file_name,
      videoUrl: file.external_url,
      embedUrl,
      isDirectVideo: directVideo,
    });
  }

  const filteredFiles = files.filter((f) =>
    f.file_name.toLowerCase().includes(search.toLowerCase())
  );

  // Right side:
  // Files assigned to CURRENT selected sidebar client + program
  const assigned = filteredFiles.filter((f) => {
    if (!clientId || !programId) return false;

    const matchesClient = f.clients.some((c) => c.id === clientId);
    const matchesProgram = f.programs.some((p) => p.id === programId);

    return matchesClient && matchesProgram;
  });

  // Left side:
  // Files NOT assigned to CURRENT selected sidebar client + program
  // Includes:
  // 1. truly unassigned files
  // 2. files assigned to some other client/program
  const available = filteredFiles.filter((f) => {
    if (!clientId || !programId) {
      // If sidebar selection is incomplete, show only truly global unassigned
      return f.clients.length === 0 && f.programs.length === 0;
    }
```

```
  const matchesClient = f.clients.some((c) => c.id === clientId);
  const matchesProgram = f.programs.some((p) => p.id === programId);

  // If already assigned to current selection, exclude from left side
  return !(matchesClient && matchesProgram);
});

function FileCard({
  file,
  isAvailableSection,
}: {
  file: Attachment;
  isAvailableSection: boolean;
}) {
  const cardRef = useRef<HTMLDivElement>(null);

  const isTrulyUnassigned =
    file.clients.length === 0 && file.programs.length === 0;

  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (
        openMenu === file.id &&
        cardRef.current &&
        !cardRef.current.contains(event.target as Node)
      ) {
        setOpenMenu(null);
      }
    }

    document.addEventListener("mousedown", handleClickOutside);
    return () => {
      document.removeEventListener("mousedown", handleClickOutside);
    };
```

```
    }, [openMenu, file.id]);

    return (
      <div
        ref={cardRef}
        className="relative border rounded-xl p-4 bg-white dark:bg-slate-900 border-slate-200 dar
      >
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-slate-400"
          onClick={() => setOpenMenu(openMenu === file.id ? null : file.id)}
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold pr-8 break-words">{file.file_name}</p>

        {file.description && (
          <DescriptionDisplay
            html={file.description}
            className="text-xs text-slate-500 dark:text-slate-400"
          />
        )}

        <p className="text-xs">
          Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
        </p>

        {file.asset_type === "file" && file.sharepoint_url ? (
          <a
            href={file.sharepoint_url}
            target="_blank"
```

```
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 dark
          >
            View File
          </a>
        ) : file.asset_type === "url" && file.external_url ? (
          isPlayableVideoUrl(file.external_url) ? (
            <button
              onClick={() => openUrlVideo(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5 text-xs rounded-lg bg-em
            >
              ▶ Play Video
            </button>
          ) : (
            <a
              href={file.external_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-emerald-600 hover:text-emerald-700 dark:text-emerald-400 da
            >
              Open URL
            </a>
          )
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}

        {openMenu === file.id && (
          <div
            className="
              absolute right-3 top-10
              w-[180px]
              bg-white dark:bg-slate-900
```

```
                border border-slate-200 dark:border-slate-700
                p-3 rounded-lg shadow-lg
                space-y-2 z-50
              "
            >
              {isAvailableSection ? (
                <>
                  <button
                    onClick={() => assignFileToCurrentSelection(file)}
                    className="w-full px-3 py-2 text-sm rounded bg-emerald-600 text-white hover:bg-
                  >
                    Assign
                  </button>

                  {/* Delete only if file is truly global unassigned */}
                  {isTrulyUnassigned && (
                    <button
                      onClick={() => requestDeleteFile(file.id)}
                      className="w-full px-3 py-2 text-sm rounded bg-red-600 text-white hover:bg-re
                    >
                      Delete
                    </button>
                  )}
                </>
              ) : (
                <button
                  onClick={() => unassignFile(file.id)}
                  className="w-full px-3 py-2 text-sm rounded bg-amber-500 text-white hover:bg-ambe
                >
                  Unassign
                </button>
              )}
            </div>
          )}
        </div>
```

```
    );
  }

  return (
    <div className="space-y-6">
      <div className="mb-4 flex justify-center">
        <div className="relative w-full max-w-md">
          <svg
            className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-slate-400 dark:text-
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
            />
          </svg>

          <input
            type="text"
            placeholder="Search files..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-10 pr-4 py-2 border border-slate-200 dark:border-slate-700 round
          />
        </div>
      </div>

      <ConfirmDialog
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
```

```
          confirmLabel="Delete"
          cancelLabel="Cancel"
          variant="danger"
          onConfirm={confirmDeleteFile}
          onCancel={() => setFileIdToDelete(null)}
        />

        <ToastContainer toasts={toasts} onClose={removeToast} />

        <UrlVideoPopup
          open={videoPopup != null}
          title={videoPopup?.title || "Video Player"}
          videoUrl={videoPopup?.videoUrl || null}
          embedUrl={videoPopup?.embedUrl || null}
          isDirectVideo={videoPopup?.isDirectVideo || false}
          onClose={() => setVideoPopup(null)}
        />

        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
          {/* Available Files */}
          <div className="bg-white dark:bg-slate-900 p-4 rounded-xl border border-slate-200 dark:bo
            <h3 className="mb-3 text-slate-900 dark:text-slate-100">
              📂 Available Files
            </h3>

            <div className="space-y-3 h-[500px] overflow-y-auto">
              {available.length > 0 ? (
                available.map((file) => (
                  <FileCard
                    key={file.id}
                    file={file}
                    isAvailableSection={true}
                  />
                ))
              ) : (
```

```
                <div className="text-center py-8 text-slate-500 dark:text-slate-400">
                  No available files match your search
                </div>
              )}
            </div>
          </div>

          {/* Assigned Files */}
          <div className="bg-white dark:bg-slate-900 p-4 rounded-xl border border-slate-200 dark:bo
            <h3 className="mb-3 text-slate-900 dark:text-slate-100">
              ✅ Assigned Files
            </h3>

            <div className="space-y-3 h-[500px] overflow-y-auto">
              {assigned.length > 0 ? (
                assigned.map((file) => (
                  <FileCard
                    key={file.id}
                    file={file}
                    isAvailableSection={false}
                  />
                ))
              ) : (
                <div className="text-center py-8 text-slate-500 dark:text-slate-400">
                  {!clientId || !programId
                    ? "Select a client and program from the sidebar to view assigned files"
                    : "No assigned files match your search"}
                </div>
              )}
            </div>
          </div>
        </div>
      </div>
    );
  }
```

✅ Important note for your current working branch

I added this in `uploadedfileslist.tsx`:

```
TypeScript
```

inside:

- `notifyAttachmentsUpdated()`

This is **good** because your `programattachments.tsx` is already listening for:

```
TypeScript
```

So now when you:

- assign
- unassign
- delete

the learner/program side refreshes more reliably.

✅ After pasting these files, test this checklist

Test 1 — file asset

- `asset_type = "file"`
- Click **View file**
- Opens SharePoint

Test 2 — normal URL

- `asset_type = "url"`
- Example: Google / docs page
- Should show **Open link / Open URL**
- Opens in new tab

Test 3 — YouTube

- Should show ▶️ **Play video**
- Opens popup

Test 4 — Vimeo

- Should show ▶️ **Play video**

Test 5 — Loom share link

- Should show ▶️ **Play video**

Test 6 — direct `.mp4`

- Should show ▶️ **Play video**

Test 7 — assign/unassign/delete

- Ensure cards move between columns correctly
- Ensure program side refreshes

✅ My recommendation (very important)

Since this video URL logic is now repeated in **3 files**, next best cleanup is:

Create reusable helper file:

```
TypeScript
```

with:

- `isDirectVideoUrl`
- `getEmbedUrl`
- `isPlayableVideoUrl`

Then all 3 components can import same logic.

This avoids future mismatch bugs.

✅ Best next step

If you want, I can now give you **one more improvement**:

I can generate a shared reusable helper file:

- `videoUtils.ts`

and then give you the **cleaner final versions** of these 3 files using that helper (much better for maintenance).

If you want that, reply:

“**give me reusable videoUtils.ts version**”

and I’ll make it clean and production-ready.

```
"use client";

import { FileText, FolderOpen } from "lucide-react";
import { useEffect, useState, useCallback } from "react";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Attachment = {
  id: number;
  file_name: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
};

export default function ProgramAttachments({
  programId,
}: {
  programId?: string | number | null;
}) {
```

```
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;
  const [files, setFiles] = useState<Attachment[]>([]);
  const [loading, setLoading] = useState(false);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  const loadFiles = useCallback(async () => {
    if (!programId) {
      setFiles([]);
      return;
    }

    setLoading(true);

    try {
      const res = await fetch(`${API}/api/attachments/?program=${programId}`);
      const data = await res.json();
      setFiles(data);
    } catch (err) {
      console.error("Failed to load program files", err);
      setFiles([]);
    } finally {
      setLoading(false);
    }
```

```
}, [programId, API]);

// Initial load + reload when program changes
useEffect(() => {
  loadFiles();
}, [loadFiles]);

// Listen for attachment updates from admin dashboard
useEffect(() => {
  function handleAttachmentsUpdated() {
    loadFiles();
  }

  function handleStorageChange(event: StorageEvent) {
    if (event.key === "attachments_updated_at") {
      loadFiles();
    }
  }

  function handleFocus() {
    loadFiles();
  }

  function handleVisibilityChange() {
    if (document.visibilityState === "visible") {
      loadFiles();
    }
  }
```

```
    window.addEventListener("attachments-updated",
handleAttachmentsUpdated);
    window.addEventListener("storage", handleStorageChange);
    window.addEventListener("focus", handleFocus);
    document.addEventListener("visibilitychange",
handleVisibilityChange);

    return () => {
      window.removeEventListener("attachments-updated",
handleAttachmentsUpdated);
      window.removeEventListener("storage", handleStorageChange);
      window.removeEventListener("focus", handleFocus);
      document.removeEventListener("visibilitychange",
handleVisibilityChange);
    };
  }, [loadFiles]);

  function isDirectVideoUrl(url: string) {
    const lower = url.toLowerCase();
    return (
      lower.endsWith(".mp4") ||
      lower.endsWith(".webm") ||
      lower.endsWith(".ogg") ||
      lower.endsWith(".mov") ||
      lower.endsWith(".m4v")
    );
  }

  function getEmbedUrl(url: string): string | null {
```

```
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? https://www.youtube.com/embed/${videoId} :
null;
    }

    // YouTube normal link
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return https://www.youtube.com/embed/${videoId};

      // already embed URL
      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? https://player.vimeo.com/video/${videoId} :
null;
    }

    // Loom
```

```
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return https://www.loom.com/embed/${pathParts[shareIndex +
1]};
      }

      const embedIndex = pathParts.findIndex((part) => part ===
"embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) return;
```

```
  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}

if (!programId) return null;

return (
  <>
    <UrlVideoPopup
      open={videoPopup != null}
      title={videoPopup?.title || "Video Player"}
      videoUrl={videoPopup?.videoUrl || null}
      embedUrl={videoPopup?.embedUrl || null}
      isDirectVideo={videoPopup?.isDirectVideo || false}
      onClose={() => setVideoPopup(null)}
    />
```

```
      <div className="flex flex-col">
        {/* Header */}
        <div className="flex items-center gap-2 mb-3">
          <div className="flex h-8 w-8 items-center justify-center
rounded-lg bg-gradient-to-br from-sky-500 to-sky-600">
            <FolderOpen className="h-4 w-4 text-white" />
          </div>
          <div>
            <h2 className="text-xs lg:text-sm font-semibold text-slate-
900 dark:text-slate-50">
              Learning Resources
            </h2>
            <p className="text-[9px] lg:text-xs text-slate-600 dark:text-
slate-400">
              Files & materials
            </p>
          </div>
        </div>

        {/* Content Box */}
        <div className="h-[100px] overflow-y-auto rounded-lg border
border-slate-200 dark:border-slate-800 bg-white dark:bg-slate-900/70
p-2.5">
          {loading ? (
            <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
              <FileText className="mx-auto h-5 w-5 text-slate-400
```

```
dark:text-slate-600" />
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
              Loading files...
            </p>
          </div>
        ) : files.length === 0 ? (
          <div className="rounded-lg border border-dashed border-
slate-300 dark:border-slate-700 bg-slate-50/50 dark:bg-slate-900/30 p-
3 text-center">
            <FileText className="mx-auto h-5 w-5 text-slate-400
dark:text-slate-600" />
            <p className="mt-2 text-[10px] lg:text-xs font-medium text-
slate-600 dark:text-slate-400">
              No files assigned to this program
            </p>
          </div>
        ) : (
          <div className="space-y-2">
            {files.map((file) => {
              const isVideoUrl =
                file.asset_type === "url" &&
                !!file.external_url &&
                isPlayableVideoUrl(file.external_url);

              return (
                <div
                  key={file.id}
                  className="rounded-lg border border-slate-200
```

```
dark:border-slate-800 bg-white dark:bg-slate-900/60 p-2"
            >
              <div className="flex items-start gap-2">
                <div className="flex h-7 w-7 flex-shrink-0 items-center
justify-center rounded-md bg-slate-100 dark:bg-slate-800 text-slate-
600 dark:text-slate-300">
                  <FileText className="h-3.5 w-3.5" />
                </div>

                <div className="min-w-0 flex-1">
                  <span className="block text-[10px] lg:text-xs font-
medium text-slate-900 dark:text-slate-100 truncate">
                    {file.file_name}
                  </span>

                  {/* Action */}
                  <div className="mt-1.5">
                    {file.asset_type === "file" && file.sharepoint_url ? (
                      <a
                        href={file.sharepoint_url}
                        target="_blank"
                        rel="noopener noreferrer"
                        className="inline-flex items-center gap-1 text-
[10px] lg:text-xs font-medium text-emerald-600 dark:text-emerald-400
hover:underline"
                      >
                        View file
                        <span aria-hidden>→</span>
                      </a>
```

```
                    ) : file.asset_type === "url" && file.external_url ? (
                      isVideoUrl ? (
                        <button
                          type="button"
                          onClick={() => openUrlVideo(file)}
                          className="inline-flex items-center gap-1 px-2.5 py-1 text-[10px] lg:text-xs font-medium rounded-lg bg-emerald-600 text-white hover:bg-emerald-700 transition"
                        >
                          ▶️ Play video
                        </button>
                      ) : (
                        <a
                          href={file.external_url}
                          target="_blank"
                          rel="noopener noreferrer"
                          className="inline-flex items-center gap-1 text-[10px] lg:text-xs font-medium text-emerald-600 dark:text-emerald-400 hover:underline"
                        >
                          Open link
                          <span aria-hidden>→</span>
                        </a>
                      )
                    ) : (
                      <span className="text-[10px] lg:text-xs text-slate-400 dark:text-slate-500">
                        No link available
                      </span>
```

```
                )}
              </div>
            </div>
          </div>
        </div>
      );
    })}
  </div>
 )}
</div>
</div>
</>
);
}
```

i want to increase height of inner rectangle

Yes 👍

In your `ProgramAttachments.tsx`, the **inner rectangle** is this div:

```
<div className="h-[100px] overflow-y-auto rounded-lg border border-slate-200 dark:border-slate-80
```

TypeScript

✅ To increase height

Just change:

TypeScript

to a bigger value.

Recommended options

Small increase

TypeScript

Medium increase

TypeScript

Larger (good for more files)

TypeScript

✅ Exact line to change

Current:

```
<div className="h-[100px] overflow-y-auto rounded-lg border border-slate-200 dark:border-slate-80
```

TypeScript

Example updated:

```
<div className="h-[180px] overflow-y-auto rounded-lg border border-slate-200 dark:border-slate-80
```

TypeScript

✅ Best recommendation for your UI

Since this is **Learning Resources** and you may have multiple files, I suggest:

TypeScript

It gives:

- better visibility
- still compact
- avoids too much scrolling

✅ If you want responsive height (better)

Instead of fixed one height for all screens, use:

```
<div className="h-[140px] lg:h-[180px] overflow-y-auto rounded-lg border border-slate-200 dark:bo
```

TypeScript

This means:

- mobile/smaller screens → `140px`
- large screens → `180px`

👉 This is usually the **best option**.

✅ Final recommended change

Use this:

```
<div className="h-[140px] lg:h-[180px] overflow-y-auto rounded-lg border border-slate-200 dark:bo
```

TypeScript

✅ If you want even better UX

If you want, I can also give you:

2 better versions of this box:

1. **Auto-expand up to max height** (looks nicer)
2. **Fixed height but visually cleaner card style**

If you want, reply:

“give me best UI version for programattachments box”

Is this conversation helpful so far?